

PROgramming in LOGic

CS 331 Tutorial # 4

Rashmi Dutta Baruah

Department of Computer Science & Engineering

IIT Guwahati

Outline

- Database Manipulation
- Collecting solutions
- Example

Database Manipulation

- Prolog has four database manipulation commands:
assert, retract, asserta, and assertz.

```
?- assert(happy(mia)).
```

```
?- assert(happy(vincent)).
```

```
?- assert( (naive(X):- happy(X)) ).
```

```
?- listing.
```

```
happy(mia).  
happy(vincent).  
naive(A):-  
    happy(A)
```

```
?- retract(happy(mia)).
```

```
?- retract(happy(X)).
```

Database Manipulation

- If we want more control over where the asserted material is placed, there are two variants of `assert/1`, namely:
 - **assertz**. Places asserted material at the end of the database.
 - **asserta**. Places asserted material at the beginning of the database.

```
:- dynamic lookup/3.  
add_and_square(X,Y,Res):-  
    lookup(X,Y,Res), !.  
  
add_and_square(X,Y,Res):-  
    Res is (X+Y)*(X+Y),  
    assert(lookup(X,Y,Res)).
```

it takes two numbers X and Y, adds X to Y, and then squares the result.

lookup/3 declared as a dynamic predicate, need to do this as we plan to change the definition of `lookup/3` during run-time.

Database Manipulation

```
?- add_and_square(3,7,X) .  
X = 100  
Yes  
?- add_and_square(3,4,Y) .  
Y = 49  
yes
```

- If we now ask for a listing we see that the database now contains
lookup(3, 7, 100) .
lookup(3, 4, 49) .
- Should we later ask Prolog to add and square 3 and 4, it wouldn't perform the calculations again. Rather, it would just return the previously calculated result.

Collecting Solutions

- There may be many solutions to a query. For example, suppose we are working with the database

```
child(martha,charlotte).  
child(charlotte,caroline).  
child(caroline,laura).  
child(laura,rose).
```

```
descend(X,Y) :- child(X,Y).
```

```
descend(X,Y) :- child(X,Z),  
                 descend(Z,Y).
```

```
?-descend(martha,X).
```

there are four solutions

(namely $X=charlotte$, $X=caroline$, $X=laura$, and $X=rose$).

- Prolog generates these solutions one by one.
- What if we would like to have all the solutions to a query, and we would like them handed to us in a neat, usable, form.
- three built-in predicates that do this: **findall**, **bagof** and **setof**

Collecting Solutions

?- **findall(Object,Goal,List)** .

- produces a list List of all the objects Object that satisfy the goal Goal (give me a list containing all the instantiations of Object which satisfy Goal) .

```
?- findall(X,descend(martha,X),Z) .  
X = _7489  
Z = [charlotte,caroline,laura,rose]
```

```
?- findall(Child,descend(Mother,Child),List) .  
Child = _6947  
Mother = _6951  
List = [charlotte,caroline,laura,rose,caroline,  
        laura,rose,laura,rose,rose]
```

Collecting Solutions

?- bagof(Child,descend(Mother,Child) ,List) .

we get the response

Child = _7736

Mother = caroline

List = [laura,rose] ;

Child = _7736

Mother = charlotte

List = [caroline,laura,rose] ;

Child = _7736

Mother = laura

List = [rose] ;

Child = _7736

Mother = martha

List = [charlotte,caroline,laura,rose] ;

no

Collecting Solutions

- **setof/3** basically the same as **bagof/3** , but with one useful difference: the lists it contains are ordered and contain no redundancies (that is, no list contains repeated items).

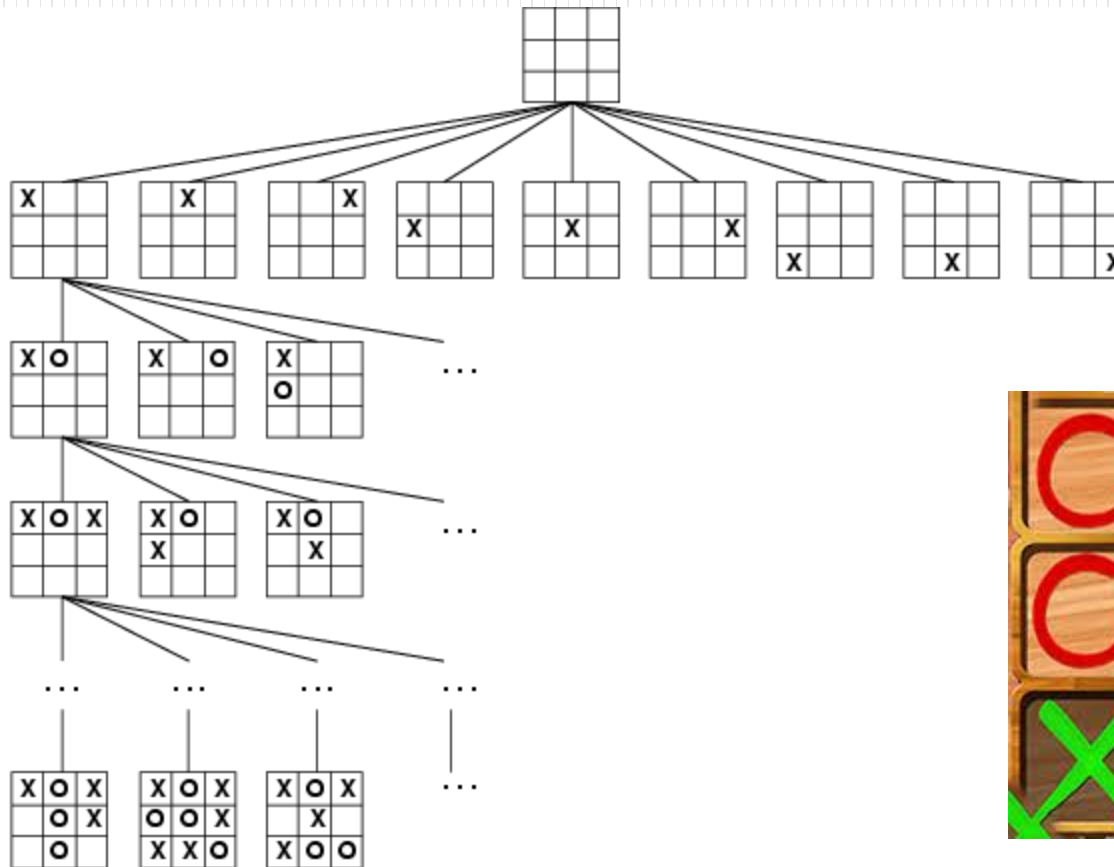
```
age(harry,13) .  
age(draco,14) .  
age(ron,13) .  
age(hermione,13) .  
age(dumbledore,60) .  
age(hagrid,30) .
```

we have to tell setof/3 not to generate separate lists for each value of X , we do this with the ^ symbol

```
?- findall(Y,age(X,Y),Out) .  
Y = _8847  
X = _8851  
Out = [13,14,13,13,60,30]
```

```
?- setof(Y,X^age(X,Y),Out) .  
Y = _8981  
X = _8985  
Out = [13,14,30,60]
```

Tic-Tac-Toe Example



Tic-Tac-Toe Example

- Board Representation

```
[A, B, C,  
 D, E, F,  
 G, H, I]
```

Each cell can be:

x → player X

o → player O (computer)

b → blank (empty)

initial board: [b, b, b, b, b, b, b, b, b]

Winning Conditions : win/2

```
win(Board, Player) :-  
    rowwin(Board, Player).  
win(Board, Player) :-  
    colwin(Board, Player).  
win(Board, Player) :-  
    diagwin(Board, Player).
```

Row wins

```
rowwin(Board, Player) :-  
    Board =  
    [Player, Player, Player, _, _, _, _, _].
```

Column wins

```
colwin(Board, Player) :-  
    Board =  
    [Player, _, _, Player, _, _, Player, _, _].
```

Diagonal wins

```
diagwin(Board, Player) :-  
    Board =  
    [Player, _, _, _, Player, _, _, _],  
    Player.
```

A player wins if they have a winning **row OR column OR diagonal**.
Prolog tries these rules **one by one** using **backtracking**.

First row has all Player , similarly for row 2 and row 3.

Matches column 1, Other clauses match column 2 and 3.

Two diagonals are checked similarly

```
other(x,o) .  
other(o,x) .
```

Helping predicate for alternating play in a "self" game. If current player is X, next is O, and vice versa.

Core Game Loop (Self-play):

```
game(Board, Player) :-  
    win(Board, Player),    !,  
    write([player, Player, wins]).
```

```
game(Board, Player) :-  
    other(Player, Otherplayer),  
    move(Board, Player, Newboard),  
    !, display(Newboard),  
    game(Newboard, Otherplayer) .
```

Checks if current player already won, ! (cut) stops further backtracking Prints the winner

Find the other player, Make a move
Display the board, Recurse with the next player. Prolog automatically explores moves via **backtracking**.

Making a Move:

```
move([b,B,C,D,E,F,G,H,I],  
    Player,  
    [Player,B,C,D,E,F,G,H,I]) .
```

If first cell is blank (b), place Player there.

There are **9 such rules**, one per position. Prolog tries moves **top to bottom**, this defines the **move order**.

Displaying the Board

```
display([A,B,C,D,E,F,G,H,I]) :-  
    write([A,B,C]), nl,  
    write([D,E,F]), nl,  
    write([G,H,I]), nl, nl.
```

Starting Self-Play

```
selfgame :-  
    game([b,b,b,b,b,b,b,b,b], x).
```

```
x_can_win_in_one(Board) :-  
    move(Board, x, Newboard),  
    win(Newboard, x).
```

```
orespond(Board, Newboard) :-  
    move(Board, o, Newboard),  
    win(Newboard, o),  
    !.
```

Priority 1: O wins immediately
Cut ensures O **takes winning move**.

Starts with empty board and Player
x

Predicates to support playing a game
with the user.

X has an immediate winning move if:
X can move That move produces a
win. used by O to **block X**.

The predicate **orespond** generates
the computer's (playing o) response
from the current Board. Computer
(O) response logic

```
orespond(Board,Newboard) :-  
    move(Board, o, Newboard),  
    not(x_can_win_in_one(Newboard)).
```

```
orespond(Board,Newboard) :-  
    move(Board, o, Newboard).
```

```
orespond(Board,Newboard) :-  
    not(member(b,Board)),  
    !,  
    write('Cats game!'), nl,  
    Newboard = Board.
```

Handling User Input (X move)

```
xmove([b,B,C,D,E,F,G,H,I], 1,  
      [x,B,C,D,E,F,G,H,I]).
```

```
xmove(Board, _, Board) :-  
    write('Illegal move. '), nl.
```

Priority 2: Block X from winning
next move

Priority 3: Any legal move

Priority 4: Draw (no blanks left)

If user enters 1, place X in cell 1 if
blank. There are 9 such rules.

Starting Human vs Computer Game

```
playo :-  
    explain,  
    playfrom([b,b,b,b,b,b,b,b,b]).
```

Explain rules

```
explain :-  
    write('You play X by  
entering integer positions  
followed by a period. '),  
    nl,  
    display([1,2,3,4,5,6,7,8,9]).
```

Shows mapping of numbers to board positions.

Main game loop

```
playfrom(Board) :-  
    win(Board, x), write('You  
win!').  
playfrom(Board) :-  
    win(Board, o), write('I  
win!').
```

Stops when someone wins.


```
playfrom(Board) :-  
    read(N),  
    xmove(Board, N, Newboard),  
    display(Newboard),  
    orespond(Newboard, Newnewboard),  
    display(Newnewboard),  
    playfrom(Newnewboard).
```

Read user move
Update board
Display
Computer responds
Display
Recurse

Summary

- In this tutorial we learnt:
 - some more built-in predicates
 - Example program for Tic-Tac-Toe game